

به نام خدا

گزارش تمرین کامپیوتری اول

سیگنال‌ها و سیستم‌ها-دکتر اخوان

امیرمحمد میرحسینی ۸۱۰۱۰۲۶۰۱

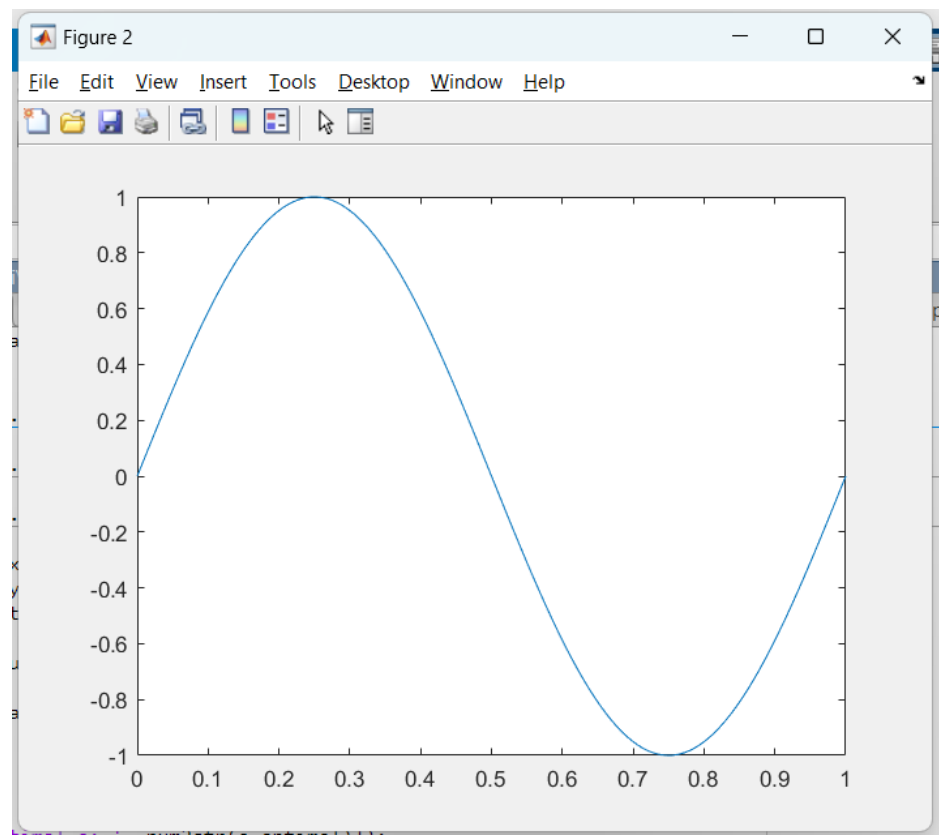
سوال ۱:

shortProblems.m , twoLinePlot.m

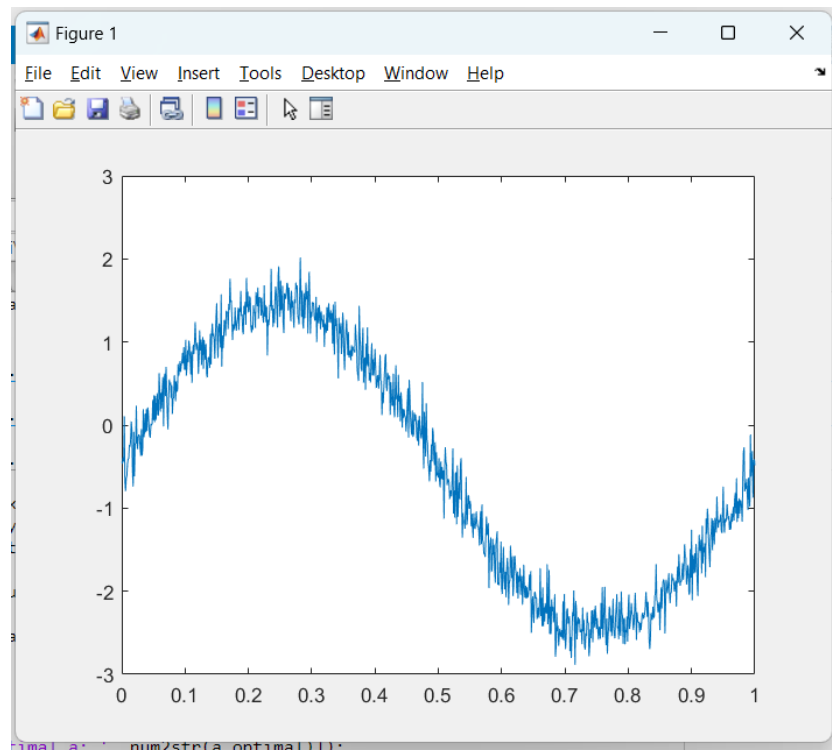
سوال ۲:

partTwo.m , p2_4.m , testFunction.m

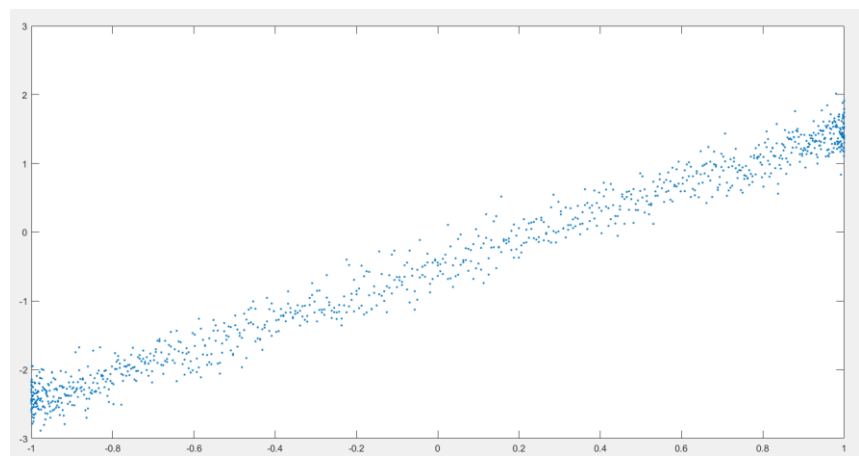
(۱-۲)



(۲-۲)

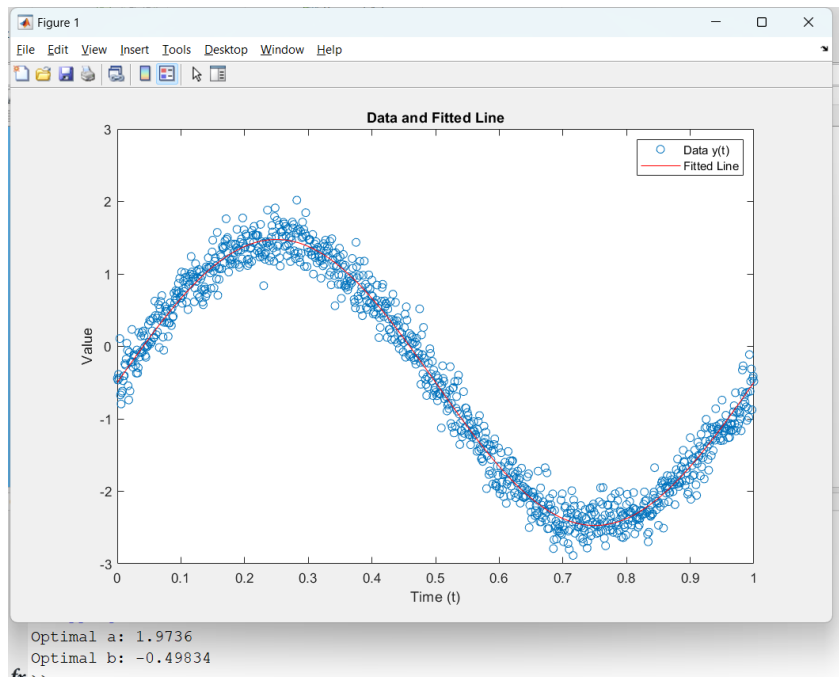


(۳-۲)



شیب این خط مقدار α را به ما می دهد و عرض از مبدا هم مقدار β را به ما می دهد.

(۴-۲)



$$\alpha = 1.9736, \beta = -0.49834$$

تست فانکشن: *testFunction.m*

برای خروجی بدون نویز دقیق، و برای خروجی با نویز هم تقریباً دقیق به دست می‌آورد (به دلیل وجود نویز آلفا و بتا تئوری را کاملاً دقیق بدست نمی‌آورد اما بسیار نزدیک است).

```

1      %% testing WITHOUT NOISE
2
3      mm=length(t);
4      x1 = rand(1,mm);
5      a1 = 6;b1 = 4;
6      y1 = a1*x1 + b1;
7      optimal_params = fsolve(@(params) p2_4(params, y1, x1), initial_guess);
8
9      a1_test = optimal_params(1);
10     b1_test = optimal_params(2);
11
12     disp(a1_test);
13     disp(b1_test);
14
15     %% testing the function WITH NOISE
16     x2 = rand(1,mm);
17     Noise = sqrt(0.3)*randn(1,mm);
18     a2 = 5;
19     b2 = 7;
20     y2 = a2*x2+b2+Noise;
21     optimal_params = fsolve(@(params) p2_4(params, y2, x2), initial_guess);
22
23     a2_test = optimal_params(1);
24     b2_test = optimal_params(2);
25     disp(a2_test);

```

Command Window

the problem appears regular as measured by the gradient.

<stopping criteria details>

6.0000

4.0000

```

14 %% testing the function WITH NOISE
15 x2 = rand(1,mm);
16 Noise = sqrt(0.3)*randn(1,mm);
17 a2 = 5;
18 b2 = 7;
19 y2 = a2*x2+b2+Noise;
20 optimal_params = fsolve(@(params) p2_4(params, y2, x2), initial_guess);
21
22 a2_test = optimal_params(1);
23 b2_test = optimal_params(2);
24 disp(a2_test);
25 disp(b2_test);
26

```

Command Window

value of the function tolerance.

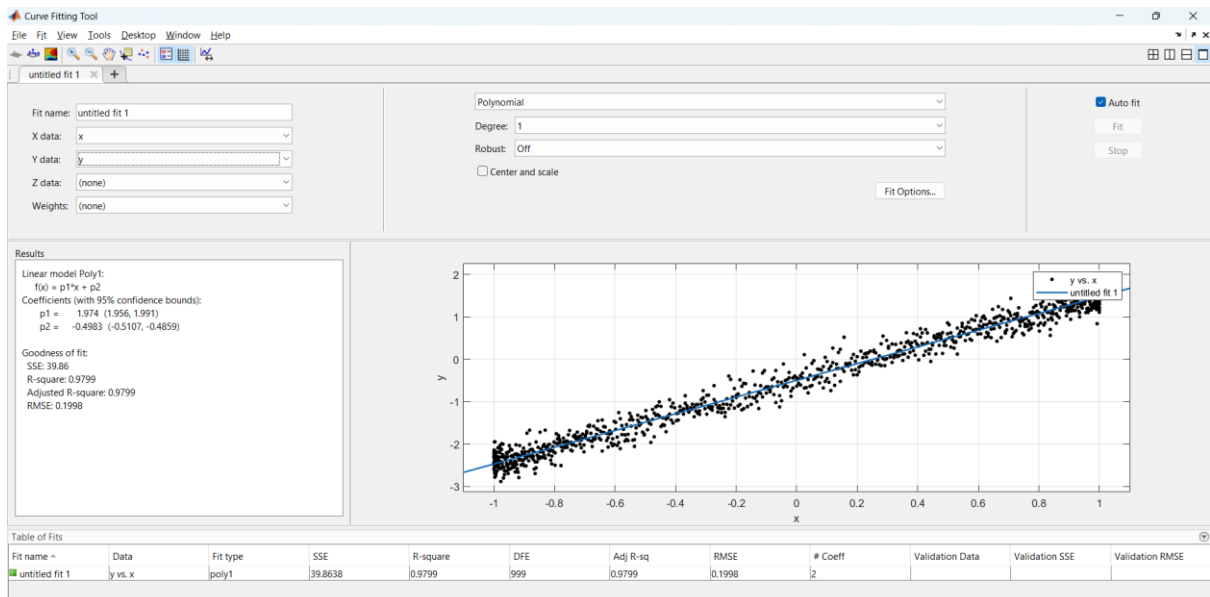
<stopping criteria details>

5.0681

6.9686

x >>

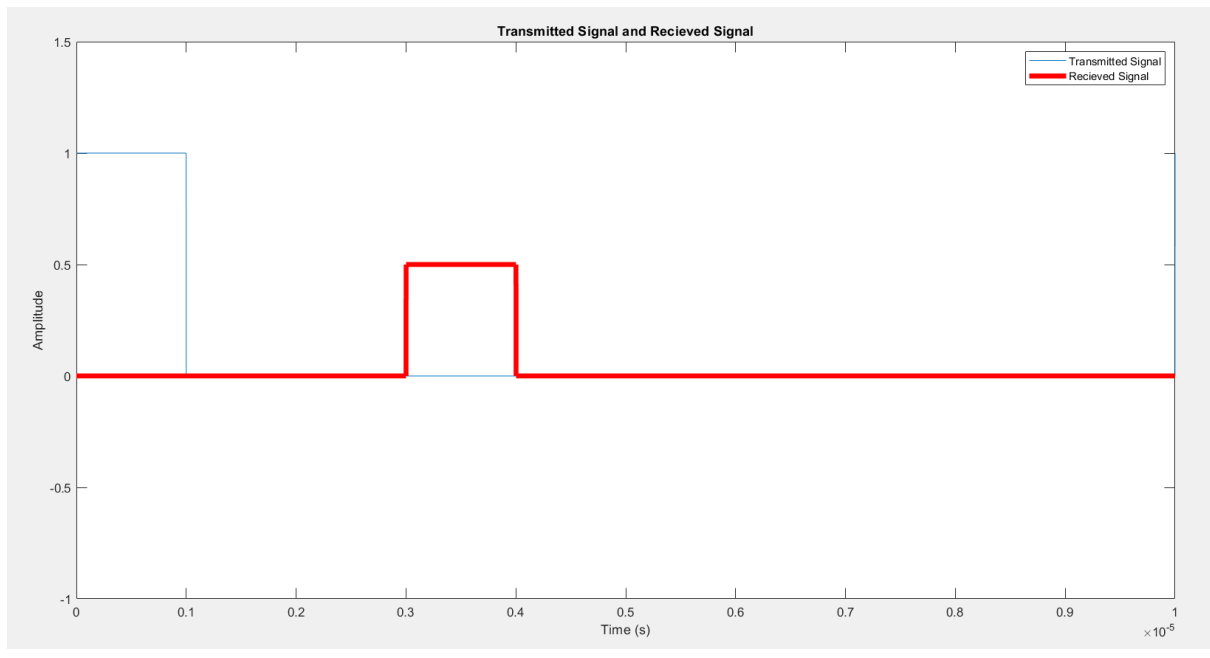
۲-۵) بله تقریباً برابر است با مقادیر به دست آمده در قسمت قبل



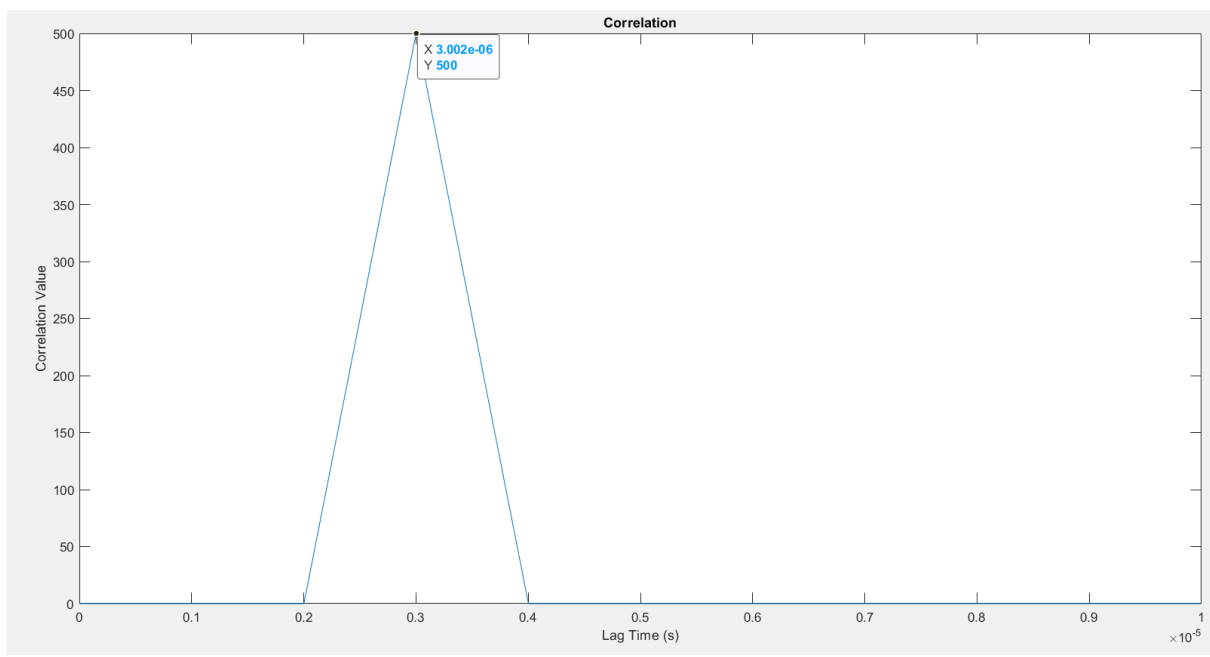
PartThree.m

۳-۲ و ۱-۳

به ترتیب سیگنال فرستاده شده و دریافتی در *legend* نمودار اشاره شده است.



(۳-۳)



تابع کورلیشن ساخته شده در انتهای فایل قرار داده شده است.

$$R = t_d C / 2$$

که با جای گذاری مقدار t_d نمودار R بدست می آید: ۴۴۹.۹۸۸۵

```

37 R_plot = td_plot * physconst('lightspeed')/2
38 R_plot
39 %%

```

Command Window

```

449.9885

R_plot =

449.9885

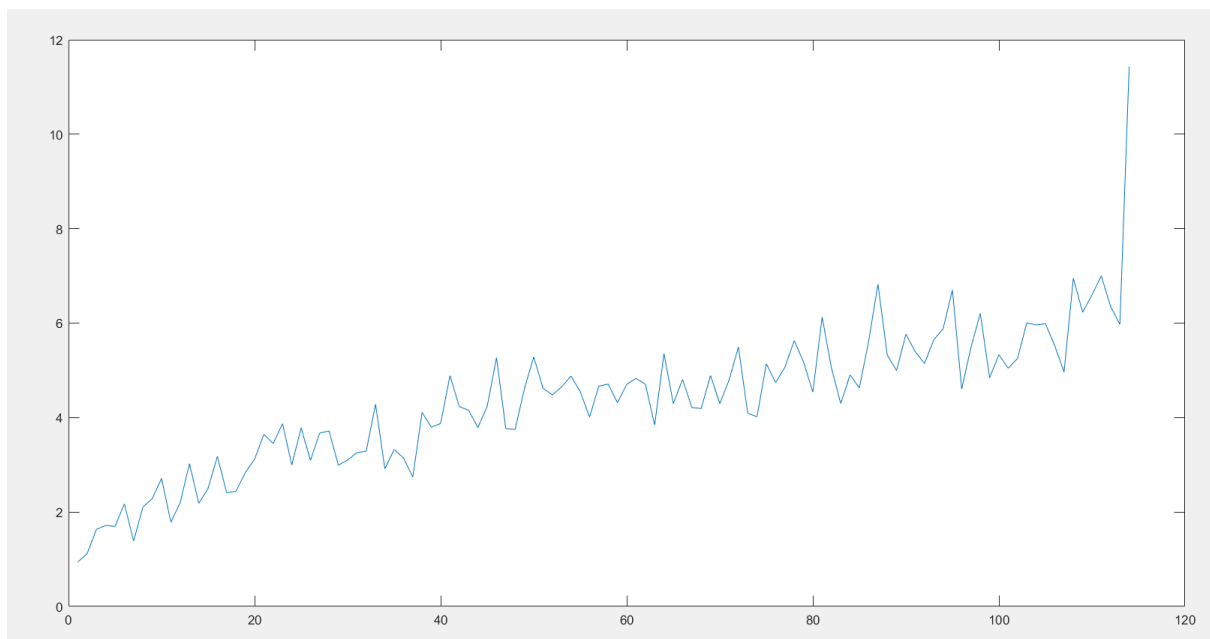
```

(۳-۴)

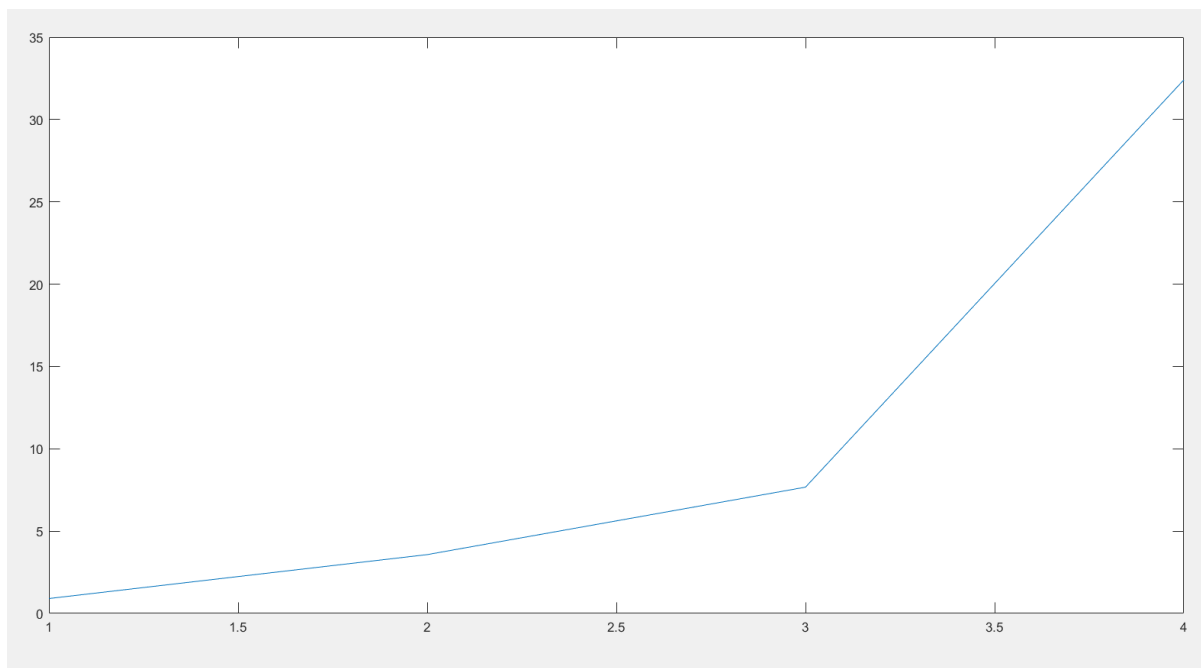
به دلیل طولانی شدن ران تایم برنامه از تابع آماده کورلیشن متلب در این بخش استفاده شده است.

در این بخش با استفاده از دو روش *nth root* و *linear*، نویز را افزایش می‌دهیم که در نمودار خروجی تاثیر به‌سزایی دارد.

افزایش *nthroot*:



افزایش خطی:



که خود استاد نیز از افزایش *nthroot* استفاده کردند.

محور عمودی: (y)

محور عمودی نشان‌دهنده مقدار خطای تخمین فاصله است.

این خطا در واحد متر محاسبه شده است.

استفاده از ریشه *nthroot* باعث می‌شود که افزایش دامنه نویز به صورت نمایی باشد. یعنی در ابتدا افزایش آهسته‌تر و در ادامه افزایش سریع‌تر اتفاق می‌افتد.

این رفتار نمایی شبیه‌تر به واقعیت است، چرا که در سیستم‌های واقعی، معمولاً نویز در سطوح پایین کم است و با افزایش سطح سیگنال، دامنه نویز نیز به صورت نمایی افزایش می‌یابد.

و به همین دلیل مقادیر یکس دو نمودار در $y=10$ متفاوت می‌باشد.

(در خطی قدرت نویز که از خطا بیشتر می‌شود تقریباً بین ۳ تا ۳.۱ و در *nthroot* تقریباً بین ۱۵۰ تا ۲۰۰)

باید توجه داشت چون از تابع رندوم استفاده شده است جواب هر دفعه متفاوت است برای همین به صورت بازه

اعلام شد. (به‌عنوان مثال یکبار ۳.۰۷۹ و یکبار ۳.۰۹۴ در خطی مقدار $y=10$ را داد)